

SQL-First Trade & Risk Analytics: Window Functions, Recursive CTEs and Manual Statistical Primitives on a Simulated Trading Book

TUNG Alexandre

EFREI Paris

Quantitative Finance — SQL & Data Engineering

September 2026

Abstract

This document describes a portfolio project in which **SQL is used as the analytical engine, not as a storage layer**. Every business calculation performed on a simulated trading book — running positions, realized and unrealized P&L, historical Value-at-Risk, rolling z-score anomaly detection, corporate-action adjustment, position reconciliation against a second source of truth, sector concentration, and client performance ranking — is implemented as a single SQL query against a SQLite database, using window functions, chained common table expressions (CTEs), a recursive CTE, and manually derived statistical primitives (percentile, standard deviation) that SQLite does not provide natively. Python is used exclusively to (1) generate a realistic synthetic market and trading dataset via a geometric Brownian motion with regime-switching volatility, and (2) execute and display SQL results with **pandas**; no business metric is computed or recomputed in **pandas/NumPy** anywhere in the project. The document also walks through a full query-optimization exercise — **EXPLAIN QUERY PLAN** before and after adding composite indexes matched to each window function's partition and order columns — and reports the measured effect on execution time on the generated database (42,000 trades, $\approx 21,000$ market-data rows), together with an honest discussion of what that measurement does and does not demonstrate at this data scale.

Contents

1	Introduction	3
2	System Architecture and Data Model	3
2.1	Entity overview	3
2.2	Schema conventions and indexing strategy	4
3	Synthetic Data Generation Methodology	4
3.1	Reference data: instruments and clients	4
3.2	Market data: regime-switching geometric Brownian motion	5
3.3	Corporate actions	5
3.4	Trade generation: a stateful day-by-day walk	5
3.5	The reconciliation ground truth: <code>declared_positions</code>	6
4	The Nine Analytical Queries	6
4.1	Query 1 — Running-total position (window function)	6
4.2	Query 2 — Realized and unrealized P&L (chained CTEs)	7

4.3	Query 3 — Historical VaR (event-time forward-fill, manual percentile)	9
4.4	Query 4 — Rolling z-score anomaly detection (manual standard deviation) . . .	11
4.5	Queries 5–6 — Corporate-action adjustment via a recursive CTE	12
4.6	Query 7 — Position reconciliation (FULL OUTER JOIN emulation)	14
4.7	Query 8 — Sector concentration (multi-level CTE aggregation)	16
4.8	Query 9 — Client performance ranking (RANK vs. DENSE_RANK)	18
5	SQLite Gaps Closed in Pure SQL	20
6	Query Optimization: EXPLAIN QUERY PLAN and Measured Timing	21
7	Report Exports: Figures and Result Tables	22
8	Honest Limitations and Scope	22
9	Conclusion	23
A	Full Database Schema	23
B	Reproducing the Results	24

1 Introduction

Most portfolio projects that pair SQL with `pandas` use the database purely as a table store: a `SELECT * FROM table` loads the data, and every actual calculation — running totals, moving averages, percentiles, standard deviations — happens afterwards in a `DataFrame`. This project inverts that division of labor. The database owns every business calculation; `pandas` and `matplotlib` are only ever allowed to format, print, and plot a result set that SQL has already fully computed.

The motivation is deliberately narrow and stated up front: this is not a claim that SQL is always the right tool for large-scale numerical work, nor an attempt to build a production risk engine. It is a demonstration, aimed at quantitative-finance, risk, and data-engineering interviews, of fluency with the parts of SQL that a `SELECT * FROM table` approach never exercises:

- window functions (`SUM() OVER`, `AVG() OVER`, `RANK()`, `DENSE_RANK()`, `LAG()`, `ROW_NUMBER()`);
- a `WITH RECURSIVE` common table expression used to compose an ordered sequence of corporate-action events into a cumulative multiplicative adjustment factor;
- statistical primitives — a percentile and a standard deviation — built from first principles in SQL because SQLite ships neither `PERCENTILE_CONT` nor a `STDEV` window function;
- query-plan-driven optimization: reading `EXPLAIN QUERY PLAN`, understanding why a temporary B-tree sort appears, and removing it with a composite index matched to the query’s partition and order columns.

Section 2 describes the data model. Section 3 describes how the synthetic market and trading data is generated, including the two features that make the analytical queries non-trivial: a Markov-switching volatility regime, and corporate actions that silently change a client’s economic position without inserting a trade row. Section 4 is the core of the document: one subsection per query, each stating the technique demonstrated and the exact SQL that implements it, together with the results actually produced by that query on the generated database. Section 5 summarizes the SQLite feature gaps closed along the way. Section 6 walks through the optimization exercise with measured before/after timings. Section 8 states this project’s scope honestly, in the same spirit as a finite-size-effects section in a theoretical derivation.

2 System Architecture and Data Model

2.1 Entity overview

The database models a simplified brokerage: a set of tradable instruments, a client base, an executed-trade blotter, a daily market-data feed, a small set of corporate actions (splits and dividends), and an independent custodian-style position feed used purely to test reconciliation logic.

Table	Grain	Rows (generated)
<code>instruments</code>	1 row / tradable instrument	40
<code>clients</code>	1 row / client	25
<code>trades</code>	1 row / executed trade	42,000
<code>market_data</code>	1 row / instrument / trading day	≈21,000 (2 yrs)
<code>corporate_actions</code>	1 row / split or dividend event	9
<code>declared_positions</code>	1 row / client / instrument (custodian snapshot)	≈994

Table 1: Database overview: table grain and generated row counts, exactly as found in `data/risk_engine.db`. All numbers are deterministic given the fixed random seed used by the generator (Section 3).

The relational structure is intentionally simple — five foreign-key relationships, no snowflake — because the complexity this project targets is in the *analytical* SQL, not in the schema design:

```
1 instruments(id, ticker, name, sector, asset_class, currency)
2 clients(id, name, client_type, opening_date)
3 trades(id, client_id -> clients, instrument_id -> instruments,
4         "timestamp", quantity, price, side)
5 market_data(instrument_id -> instruments, "timestamp", close_price, volume)
6 corporate_actions(id, instrument_id -> instruments, date, action_type, ratio)
7 declared_positions(client_id, instrument_id, reference_date, declared_quantity)
```

Listing 1: Logical schema (see Appendix A for the full DDL).

2.2 Schema conventions and indexing strategy

Two conventions matter for everything that follows. First, `trades` and `market_data` both use `"timestamp"` as a quoted column name (a day-granularity ISO date), since `timestamp` is a reserved-adjacent identifier in SQLite. Second — and this is the foundation of Section 6 — the database is created in two stages. `generate_data.py` executes only the *table* section of `schema.sql`, with no secondary indexes beyond the implicit ones SQLite creates for primary keys. This is the deliberate “before optimization” baseline. A second, clearly delimited section of `schema.sql` defines four composite indexes, applied later and live by `analysis.py` as part of the optimization walkthrough:

```
1 CREATE INDEX IF NOT EXISTS idx_trades_client_instrument_ts
2   ON trades (client_id, instrument_id, "timestamp");
3
4 CREATE INDEX IF NOT EXISTS idx_trades_instrument_ts
5   ON trades (instrument_id, "timestamp");
6
7 CREATE INDEX IF NOT EXISTS idx_corporate_actions_instrument_date
8   ON corporate_actions (instrument_id, date);
9
10 CREATE INDEX IF NOT EXISTS idx_market_data_instrument_ts
11   ON market_data (instrument_id, "timestamp");
```

Listing 2: The four optimized indexes (`schema.sql`, section 2).

Each index is matched, column for column, to the `PARTITION BY ... ORDER BY ...` clause of the window function it is meant to serve — this pairing is the entire idea behind Section 6 and is stated explicitly here so the connection is visible before the queries themselves are introduced.

3 Synthetic Data Generation Methodology

All data is generated by `generate_data.py`, which contains no business logic: it only creates instruments, clients, prices, trades, and corporate actions, and seeds `declared_positions` with a deliberately corrupted copy of the truth for the reconciliation query. The single exception is documented explicitly in the source: seeding `declared_positions` reuses the exact same recursive-CTE adjustment logic as Queries 5–6 (Section 4.5), because that is the only correct way to compute the “true” position a custodian feed should have reported before it is perturbed. Everything is seeded from a fixed random seed (`RANDOM_SEED = 42`), so every number reported in this document is exactly reproducible by re-running the two scripts.

3.1 Reference data: instruments and clients

40 instruments are spread evenly across 10 sectors (Technology, Healthcare, Financials, Energy, Consumer Discretionary, Consumer Staples, Industrials, Materials, Utilities, Telecommunica-

tions) and three asset classes — Equity (70%), ETF (15%), Bond (15%) — each with asset-class-appropriate starting price, annual drift, and annual volatility ranges. 25 clients are split Individual (55%), Corporate (25%), Institutional (20%); roughly 55% of clients are deliberately marked *concentrated* on one home sector (an $8\times$ trading-weight bias toward that sector) so that the sector-concentration query (Section 4.7) produces genuinely contrasted, non-trivial results rather than a flat distribution.

3.2 Market data: regime-switching geometric Brownian motion

Each instrument’s daily closing price follows a discretized geometric Brownian motion,

$$S_{t+1} = S_t \exp\left[\left(\mu_d - \frac{1}{2}\sigma_t^2\right) + \sigma_t z_t\right], \quad z_t \sim \mathcal{N}(0, 1), \quad (1)$$

where μ_d is the instrument’s daily drift (annual drift divided by 252) and σ_t is a *time-varying* daily volatility. Flat i.i.d. noise would make every downstream risk query trivial, so σ_t is driven by a three-state Markov-switching volatility regime (calm $\times 0.6$, normal $\times 1.0$, turbulent $\times 2.0$) with 97% daily persistence: the regime only flips with 3% daily probability, which is enough to make realized volatility cluster in time instead of behaving like flat white noise. Daily volume is scaled by asset class and bumped on high-|return| days, plus multiplicative lognormal noise.

3.3 Corporate actions

Nine corporate-action events are generated (Table 2): six SPLIT events across five distinct instruments, and three DIVIDEND events on three further instruments. One instrument — NORIN2 — is deliberately given a *second*, later split (two successive 0.5-for-1 reverse splits, compounding to a cumulative factor of 0.25), so that the recursive CTE of Section 4.5 has a genuine multi-factor composition to exercise rather than only ever multiplying a single ratio. Ratios are not all “clean” forward splits either: MERIN1 is itself a 0.5-for-1 reverse split, which exercises the adjustment formula in the direction that *shrinks* a historical share count rather than inflating it.

Ticker	Date	Type	Ratio
AURHO3	2026-01-01	DIVIDEND	—
SILIN3	2025-10-10	DIVIDEND	—
REDHO9	2025-09-04	DIVIDEND	—
FALTE3	2024-10-31	SPLIT	3.0
MERIN1	2025-08-22	SPLIT	0.5 (reverse)
NORIN2	2026-04-28	SPLIT	0.5 (1st of two)
NORIN2	2026-05-14	SPLIT	0.5 (2nd, cumulative 0.25)
ANCSA3	2025-05-28	SPLIT	1.5
ZENCA1	2025-07-25	SPLIT	1.5

Table 2: The 9 generated corporate actions, exactly as returned by Query 5 (`corporate_action_factors.csv`).

A split is applied as a real multiplicative discontinuity in the simulated price path on the action date — exactly as an unadjusted raw price series would show it — rather than as a smoothed or after-the-fact adjustment.

3.4 Trade generation: a stateful day-by-day walk

Trades are generated by walking the trading calendar day by day while maintaining a real per-(client, instrument) holdings ledger, so that a SELL can never exceed current holdings.

Daily trade volume is deliberately non-uniform: a monthly seasonality multiplier (a quiet August at $0.55\times$, an active January at $1.30\times$), a mild day-of-week effect, and ten randomly placed “burst weeks” (4–9 trading days each, $2\text{--}4\times$ elevated activity) meant to emulate earnings season or a stress episode. Each simulated trade samples a client (activity-weighted by client type: Institutional trades roughly $5\times$ more than Individual) and an instrument (weighted $8\times$ toward a concentrated client’s home sector), with a lognormal lot size and an execution price equal to that day’s close price plus small i.i.d. execution noise.

The mechanism that makes this dataset analytically interesting is this: *when a split event fires on a given date, every affected holder’s economic position is multiplied by the split ratio directly in the holdings ledger, with no corresponding row inserted into `trades`* — exactly as a real split would behave in practice. This is precisely why the raw running-total position computed by Query 1 (Section 4.1) and the corporate-action-adjusted position computed by Query 6 (Section 4.5) genuinely diverge, rather than being reconciled for a purely cosmetic reason. Figure 2 shows this divergence for the client/instrument pair it is most visible on.

3.5 The reconciliation ground truth: `declared_positions`

`declared_positions` is seeded from the *correct* corporate-action-adjusted position for every (`client`, `instrument`) pair with a positive holding (≈ 994 rows), computed in SQL using the same recursive-CTE logic as Section 4.5. That baseline is then deliberately corrupted to simulate a realistic custodian reconciliation break: 6 rows have their quantity perturbed by a $\pm 10\text{--}15\%$ (or halved) factor, 4 rows are dropped entirely (simulating a position missing at the custodian), and 4 phantom rows are inserted for (`client`, `instrument`) pairs the client never actually traded. The 14 breaks this creates ($6 + 4 + 4$) are exactly what Query 7 (Section 4.6) is designed to detect — no more, no fewer, as confirmed in Section 4.6 on the actual generated database.

4 The Nine Analytical Queries

Each subsection below states the SQL technique the query is meant to demonstrate, gives the exact query from the `queries/` directory, and reports the results it actually produces on `data/risk_engine.db`. The recursive corporate-action-factor CTE that underlies Queries 5–8 is written out once, in full, in Section 4.5; later queries reuse the identical CTE block rather than repeating a fresh explanation of it, exactly as the query files themselves do.

4.1 Query 1 — Running-total position (window function)

Technique 1. A single `SUM() OVER (PARTITION BY client_id, instrument_id ORDER BY "timestamp")` reconstructs the cumulative position of every client/instrument book directly from the trade blotter, with zero procedural code and no self-join. `BUY` and `SELL` are first folded into one signed quantity with a `CASE` expression, then window-summed.

```

1 SELECT
2     t.client_id,
3     c.name                                     AS client_name
4     ,
5     t.instrument_id,
6     i.ticker,
7     t."timestamp"                             AS trade_date,
8     t.side,
9     t.quantity,
10    CASE WHEN t.side = 'BUY' THEN t.quantity ELSE -t.quantity END AS
11    signed_quantity,
12    SUM(CASE WHEN t.side = 'BUY' THEN t.quantity ELSE -t.quantity END)
13    OVER (

```

```

12         PARTITION BY t.client_id, t.instrument_id
13         ORDER BY t."timestamp", t.id
14         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
15     )
16     running_position
17 FROM trades t
18 JOIN clients c ON c.id = t.client_id
19 JOIN instruments i ON i.id = t.instrument_id
20 ORDER BY t.client_id, t.instrument_id, t."timestamp", t.id;

```

Listing 3: 01_position_running_total.sql — running-total position per client/instrument book.

This query returns one row per trade — 42,000 rows on the generated database, exactly matching the row count of `trades` — and is the query whose plan Section 6 optimizes first.

4.2 Query 2 — Realized and unrealized P&L (chained CTEs)

Technique 2. A chain of eight CTEs implements the moving-average-cost P&L method used by most custodians: every BUY updates the average acquisition cost of the position; a SELL does *not* change the average cost of the remaining shares, it only realizes P&L against it; the position still open today carries a latent (unrealized) P&L against the latest market price. The whole pipeline is window functions and aggregation — no correlated subquery, no procedural loop.

```

1 WITH trade_costing AS (
2     SELECT
3         t.id, t.client_id, t.instrument_id, t."timestamp", t.side, t.quantity, t
4         .price,
5         -- Running totals restricted to BUY trades: SELL rows contribute 0,
6         -- so the average cost only ever moves on a BUY (moving-average-cost
7         -- convention).
8         SUM(CASE WHEN t.side = 'BUY' THEN t.quantity ELSE 0 END)
9         OVER (PARTITION BY t.client_id, t.instrument_id
10              ORDER BY t."timestamp", t.id
11              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
12         cum_buy_qty,
13         SUM(CASE WHEN t.side = 'BUY' THEN t.quantity * t.price ELSE 0 END)
14         OVER (PARTITION BY t.client_id, t.instrument_id
15              ORDER BY t."timestamp", t.id
16              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
17         cum_buy_cost,
18         SUM(CASE WHEN t.side = 'BUY' THEN t.quantity ELSE -t.quantity END)
19         OVER (PARTITION BY t.client_id, t.instrument_id
20              ORDER BY t."timestamp", t.id
21              ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
22         position_after_trade
23     FROM trades t
24 ),
25 average_cost AS (
26     -- CTE 1: average acquisition cost of the position at each point in time.
27     SELECT tc.*, tc.cum_buy_cost / NULLIF(tc.cum_buy_qty, 0) AS
28         avg_acquisition_cost
29     FROM trade_costing tc
30 ),
31 realized_pnl_per_trade AS (
32     -- Realized P&L crystallizes on every SELL trade.
33     SELECT client_id, instrument_id,
34         CASE WHEN side = 'SELL' THEN (price - avg_acquisition_cost) *
35             quantity ELSE 0 END AS realized_pnl
36     FROM average_cost
37 ),

```

```

32 realized_pnl_total AS (
33     SELECT client_id, instrument_id, SUM(realized_pnl) AS realized_pnl_total
34     FROM realized_pnl_per_trade
35     GROUP BY client_id, instrument_id
36 ),
37 last_position AS (
38     -- CTE 2: the most recent trade per (client, instrument) tells us the
39     -- currently open quantity and the average cost carried on it.
40     SELECT client_id, instrument_id, position_after_trade AS open_position,
41            avg_acquisition_cost
42     FROM (
43         SELECT ac.*,
44                ROW_NUMBER() OVER (PARTITION BY client_id, instrument_id
45                                   ORDER BY "timestamp" DESC, id DESC) AS rn
46         FROM average_cost ac
47     ) WHERE rn = 1
48 ),
49 current_market_value AS (
50     -- CTE 3: latest available close price per instrument.
51     SELECT instrument_id, close_price AS current_price
52     FROM (
53         SELECT instrument_id, close_price,
54                ROW_NUMBER() OVER (PARTITION BY instrument_id ORDER BY "timestamp"
55                                   DESC) AS rn
56         FROM market_data
57     ) WHERE rn = 1
58 ),
59 unrealized_pnl AS (
60     SELECT lp.client_id, lp.instrument_id, lp.open_position, lp.
61            avg_acquisition_cost,
62            cmv.current_price,
63            (cmv.current_price - lp.avg_acquisition_cost) * lp.open_position AS
64            unrealized_pnl
65     FROM last_position lp
66     JOIN current_market_value cmv ON cmv.instrument_id = lp.instrument_id
67     WHERE lp.open_position > 0.0001
68 ),
69 all_pairs AS (SELECT DISTINCT client_id, instrument_id FROM trades)
70 SELECT
71     ap.client_id, c.name AS client_name, ap.instrument_id, i.ticker,
72     ROUND(COALESCE(up.open_position, 0), 2) AS open_position,
73     ROUND(up.avg_acquisition_cost, 4) AS avg_acquisition_cost,
74     ROUND(up.current_price, 4) AS current_price,
75     ROUND(COALESCE(up.unrealized_pnl, 0), 2) AS unrealized_pnl,
76     ROUND(COALESCE(rpt.realized_pnl_total, 0), 2) AS realized_pnl,
77     ROUND(COALESCE(up.unrealized_pnl, 0) + COALESCE(rpt.realized_pnl_total, 0),
78            2) AS total_pnl
79 FROM all_pairs ap
80 JOIN clients c ON c.id = ap.client_id
81 JOIN instruments i ON i.id = ap.instrument_id
82 LEFT JOIN unrealized_pnl up ON up.client_id = ap.client_id AND up.
83 instrument_id = ap.instrument_id
84 LEFT JOIN realized_pnl_total rpt ON rpt.client_id = ap.client_id AND rpt.
85 instrument_id = ap.instrument_id
86 ORDER BY total_pnl DESC;

```

Listing 4: 02_pnl_realized_unrealized.sql — realized and unrealized P&L via chained CTEs.

Remark 1. On the generated database this query returns 1,000 client/instrument rows. The largest single position by total P&L is Ridgeline Holdings (Institutional) on MERIN1: an open position of 27,210 shares at an average acquisition cost of \$491.93 against a current price of \$748.74, for \$6,987,692 unrealized plus \$3,556,897 realized — \$10,544,589 total. This is not a

coincidence: MERIN1 is the reverse-split instrument of Table 2, and the average acquisition cost is computed on raw (unadjusted) trade prices, so a large, long-held position in a name that has since undergone a corporate action naturally dominates the P&L ranking.

4.3 Query 3 — Historical VaR (event-time forward-fill, manual percentile)

Technique 3. A 95% one-day historical VaR requires a daily portfolio value even on days a client made no trade. Rather than one correlated subquery per calendar day, trade events (signed quantity deltas) are UNION ALL'd with zero-delta “marker” rows for every (client, instrument, calendar day), and a single running SUM() ordered by (event_date, is_marker) forward-fills the position across the whole calendar — the is_marker discriminator guarantees that a trade dated on a given day settles *before* that day’s marker is read, so the marker always sees the correct end-of-day position. Daily returns come from LAG(); the trailing-250-day 95% VaR is then a manual percentile, sorting returns with ROW_NUMBER() and selecting the rank matching floor(0.05 N)+1, since SQLite has no PERCENTILE.CONT/PERCENTILE.DISC (no ordered-set aggregate agg(...) WITHIN GROUP (ORDER BY ...) syntax).

```

1 WITH calendar_window AS (
2     -- The 250 most recent trading days present in market_data.
3     SELECT DISTINCT "timestamp" AS calendar_date
4     FROM market_data
5     ORDER BY calendar_date DESC
6     LIMIT 250
7 ),
8 client_instrument_pairs AS (
9     SELECT DISTINCT client_id, instrument_id FROM trades
10 ),
11 events AS (
12     -- Real trades: signed quantity delta on their execution date.
13     SELECT client_id, instrument_id, "timestamp" AS event_date,
14         CASE WHEN side = 'BUY' THEN quantity ELSE -quantity END AS delta_qty,
15         0 AS is_marker
16     FROM trades
17     UNION ALL
18     -- One zero-delta marker per (client, instrument, calendar day) in the
19     -- VaR window: this is where we read the as-of position back out.
20     SELECT p.client_id, p.instrument_id, cw.calendar_date AS event_date, 0 AS
21         delta_qty, 1 AS is_marker
22     FROM client_instrument_pairs p
23     CROSS JOIN calendar_window cw
24 ),
25 running_position AS (
26     SELECT client_id, instrument_id, event_date, is_marker,
27         SUM(delta_qty) OVER (
28             PARTITION BY client_id, instrument_id
29             ORDER BY event_date, is_marker -- trades (0) settle before the
30             marker (1) on the same day
31             ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
32         ) AS position_qty
33     FROM events
34 ),
35 position_by_day AS (
36     SELECT client_id, instrument_id, event_date AS calendar_date, position_qty
37     FROM running_position WHERE is_marker = 1
38 ),
39 portfolio_value AS (
40     SELECT pd.client_id, pd.calendar_date, SUM(pd.position_qty * md.close_price)
41     AS total_value
42     FROM position_by_day pd

```

```

40 JOIN market_data md ON md.instrument_id = pd.instrument_id AND md."timestamp
    " = pd.calendar_date
41 GROUP BY pd.client_id, pd.calendar_date
42 ),
43 daily_returns AS (
44     SELECT client_id, calendar_date, total_value,
45           (total_value - LAG(total_value) OVER (PARTITION BY client_id ORDER BY
46           calendar_date))
47           / NULLIF(LAG(total_value) OVER (PARTITION BY client_id ORDER BY
48           calendar_date), 0) AS daily_return
49     FROM portfolio_value
50 ),
51 ranked_returns AS (
52     SELECT client_id, daily_return,
53           ROW_NUMBER() OVER (PARTITION BY client_id ORDER BY daily_return ASC) AS
54           ascending_rank,
55           COUNT(*) OVER (PARTITION BY client_id) AS
56           num_observations
57     FROM daily_returns WHERE daily_return IS NOT NULL
58 )
59 SELECT
60     rr.client_id, c.name AS client_name, rr.num_observations,
61     ROUND(rr.daily_return, 6) AS var_95_daily_return,
62     ROUND(-rr.daily_return * 100, 3) AS var_95_pct_loss
63 FROM ranked_returns rr
64 JOIN clients c ON c.id = rr.client_id
65 WHERE rr.ascending_rank = CAST(0.05 * rr.num_observations AS INTEGER) + 1
66 ORDER BY var_95_pct_loss DESC;

```

Listing 5: 03_var_historical.sql — historical VaR (95%), trailing 250-trading-day window.

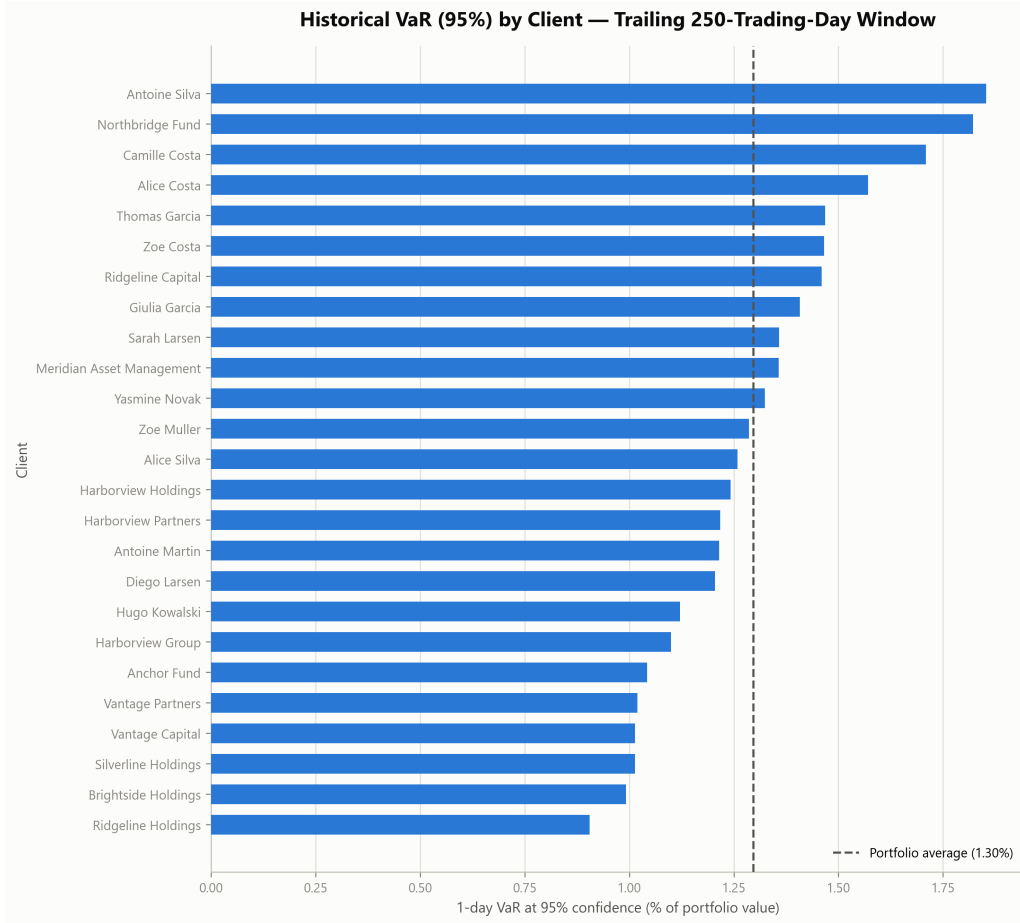


Figure 1: Historical VaR (95%) by client, 249 observations each over the trailing 250-trading-day window. Antoine Silva is the riskiest client at 1.853% of portfolio value, Ridgeline Holdings the least at 0.905%; the 25-client average is 1.297%, matching the dashed reference line.

4.4 Query 4 — Rolling z-score anomaly detection (manual standard deviation)

Technique 4. A trade is flagged as a price anomaly when it sits more than 3 standard deviations from a 20-trade rolling mean, partitioned per instrument and ordered in trade order (not calendar order, since several trades can happen on the same day). The rolling mean uses a bounded window frame (`ROWS BETWEEN 19 PRECEDING AND CURRENT ROW`); the rolling standard deviation is derived manually from $\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$ — the rolling average of squared prices minus the square of the rolling average — since SQLite has no `STDEV`/`STDEV_POP`/`STDEV_SAMP` window function. `MAX(variance, 0)` guards against a tiny negative value from floating-point rounding when the true variance is near zero, which would otherwise send `SQRT()` to `NULL`.

```

1 WITH rolling_stats AS (
2     SELECT
3         t.id AS trade_id, t.client_id, t.instrument_id, i.ticker, t."timestamp"
4         AS trade_date, t.price,
5         COUNT(*) OVER w AS window_observations,
6         AVG(t.price) OVER w AS rolling_mean,
7         AVG(t.price * t.price) OVER w AS rolling_mean_of_squares
8 FROM trades t
9 JOIN instruments i ON i.id = t.instrument_id
10 WINDOW w AS (
    PARTITION BY t.instrument_id ORDER BY t."timestamp", t.id

```

```

11         ROWS BETWEEN 19 PRECEDING AND CURRENT ROW    -- rolling window of 20
12         trades
13     ),
14     zscores AS (
15         SELECT *, SQRT(MAX(rolling_mean_of_squares - rolling_mean * rolling_mean,
16             0.0)) AS rolling_stddev
17         FROM rolling_stats
18     ),
19     zscores_final AS (
20         SELECT *, (price - rolling_mean) / NULLIF(rolling_stddev, 0) AS z_score
21         FROM zscores
22         WHERE window_observations >= 10    -- require enough history before judging a
23             trade "abnormal"
24     )
25 SELECT
26     trade_id, client_id, instrument_id, ticker, trade_date,
27     ROUND(price, 4) AS price, ROUND(rolling_mean, 4) AS rolling_mean_20t,
28     ROUND(rolling_stddev, 4) AS rolling_stddev_20t, ROUND(z_score, 3) AS z_score
29     ,
30     CASE WHEN ABS(z_score) > 3 THEN 1 ELSE 0 END AS is_anomaly
31 FROM zscores_final
32 WHERE ABS(z_score) > 3
33 ORDER BY ABS(z_score) DESC;

```

Listing 6: 04_zscore_anomaly_detection.sql — rolling z-score price-anomaly detection.

Remark 2. 546 trades are flagged out of 42,000 (1.3%). The single most extreme anomaly in the entire dataset ($z = 4.353$) is a trade in MERIN1 on **2025-08-22** — exactly the date of that instrument’s 0.5-for-1 reverse split (Table 2), which mechanically doubles the raw price overnight. This is a genuine, useful finding about the *limits* of this query rather than a data bug: Query 4 scores raw, unadjusted trade prices, so it cannot distinguish a real price dislocation from a mechanical corporate-action jump. A production anomaly detector would need to score prices adjusted by the same recursive factor used in Queries 5–6 before flagging outliers. This is stated explicitly in Section 8 rather than silently left as a quirk of the results.

4.5 Queries 5–6 — Corporate-action adjustment via a recursive CTE

Technique 5. A single WITH RECURSIVE CTE composes an ordered sequence of split/dividend events, per instrument, into a cumulative multiplicative adjustment factor: the anchor member is each instrument’s first corporate action, and the recursive member joins the next action (`rn = previous rn + 1`) and multiplies its factor into the running product — so an instrument with two successive splits (NORIN2, Table 2) correctly compounds both ratios into $0.5 \times 0.5 = 0.25$ rather than only reflecting the latest one. Query 6 then joins that factor back onto every historical trade: a trade executed *before* a split must be scaled by every split ratio that happened *after* it; a trade executed after the last split needs no adjustment (factor = 1).

```

1 WITH RECURSIVE ca_ordered AS (
2     SELECT
3         instrument_id, date, action_type,
4         CASE WHEN action_type = 'SPLIT' THEN ratio ELSE 1.0 END AS event_factor,
5         ROW_NUMBER() OVER (PARTITION BY instrument_id ORDER BY date) AS rn
6     FROM corporate_actions
7 ),
8     cumulative_factor AS (
9         -- Anchor: first corporate action of each instrument.
10        SELECT instrument_id, date, action_type, event_factor, rn, event_factor AS
11            cumulative_factor
12        FROM ca_ordered

```

```

12 WHERE rn = 1
13
14 UNION ALL
15
16 -- Recursive step: chain to the next action for the same instrument,
17 -- multiplying its factor into the running cumulative product.
18 SELECT o.instrument_id, o.date, o.action_type, o.event_factor, o.rn,
19        c.cumulative_factor * o.event_factor AS cumulative_factor
20 FROM ca_ordered o
21 JOIN cumulative_factor c ON o.instrument_id = c.instrument_id AND o.rn = c.
    rn + 1
22 )
23 SELECT cf.instrument_id, i.ticker, cf.date, cf.action_type, cf.event_factor,
24        cf.cumulative_factor AS cumulative_factor_as_of_date
25 FROM cumulative_factor cf
26 JOIN instruments i ON i.id = cf.instrument_id
27 ORDER BY cf.instrument_id, cf.date;

```

Listing 7: 05_corporate_actions_recursive_factor.sql — cumulative corporate-action adjustment factor.

```

1 WITH RECURSIVE ca_ordered AS ( /* identical to Query 5 */ ),
2 cumulative_factor AS ( /* identical to Query 5 */ ),
3 factor_total AS (
4     -- Final cumulative factor per instrument = product of ALL its corporate
5     -- actions.
6     SELECT cf.instrument_id, cf.cumulative_factor AS total_factor
7     FROM cumulative_factor cf
8     JOIN (SELECT instrument_id, MAX(rn) AS rn_max FROM ca_ordered GROUP BY
9           instrument_id) last_evt
10    ON last_evt.instrument_id = cf.instrument_id AND last_evt.rn_max = cf.rn
11 ),
12 adjusted_trades AS (
13     SELECT
14         t.id AS trade_id, t.client_id, t.instrument_id, t."timestamp" AS
15         trade_date,
16         t.side, t.quantity AS raw_quantity,
17         -- Conditional join: instruments with no corporate action at all get
18         -- a neutral factor of 1.0 via COALESCE, instead of being dropped.
19         COALESCE(ft.total_factor, 1.0) / COALESCE(
20             (SELECT cf2.cumulative_factor FROM cumulative_factor cf2
21              WHERE cf2.instrument_id = t.instrument_id AND cf2.date <= t."
22                timestamp"
23                ORDER BY cf2.date DESC LIMIT 1),
24             1.0
25         ) AS adjustment_factor
26     FROM trades t
27     LEFT JOIN factor_total ft ON ft.instrument_id = t.instrument_id
28 )
29 SELECT
30     trade_id, client_id, instrument_id, trade_date, side, raw_quantity,
31     ROUND(adjustment_factor, 6) AS adjustment_factor,
32     ROUND(raw_quantity * adjustment_factor, 4) AS adjusted_quantity,
33     ROUND(
34         SUM(CASE WHEN side = 'BUY' THEN raw_quantity * adjustment_factor
35              ELSE -raw_quantity * adjustment_factor END)
36         OVER (PARTITION BY client_id, instrument_id ORDER BY trade_date,
37               trade_id
38               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW), 4
39     ) AS running_adjusted_position
40 FROM adjusted_trades
41 ORDER BY client_id, instrument_id, trade_date, trade_id;

```

Listing 8: 06_position_adjusted_corporate_actions.sql — trade quantities restated for corporate actions.

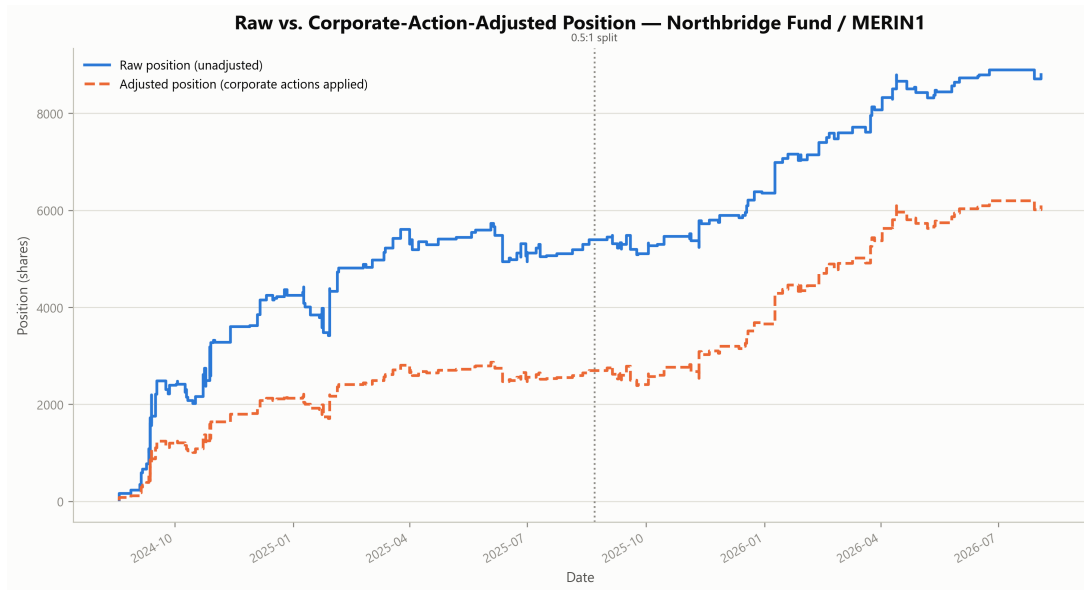


Figure 2: Raw (Query 1) vs. corporate-action-adjusted (Query 6) running position for Northbridge Fund / MERIN1, the client/instrument pair with the most trades straddling a split. The vertical dotted line marks the 0.5-for-1 reverse split of 2025-08-22: the adjusted line sits at roughly half the raw line before that date and tracks it exactly after — precisely the economic effect of Section 3’s holdings-ledger multiplication with no corresponding trade row.

4.6 Query 7 — Position reconciliation (FULL OUTER JOIN emulation)

Technique 6. SQLite has no FULL OUTER JOIN. The classic portable emulation — two LEFT JOINS combined with UNION — catches breaks on both sides of the comparison in a single pass: positions calculated internally but never declared by the custodian, positions declared by the custodian with no supporting trade history (phantoms), and positions present on both sides but with a mismatched quantity.

```

1 WITH RECURSIVE ca_ordered AS ( /* as in Query 5 */ ),
2 cumulative_factor AS ( /* as in Query 5 */ ),
3 factor_total AS ( /* as in Query 6 */ ),
4 calculated_position AS (
5     -- Our internal, corporate-action-adjusted position per (client, instrument)
6     -- same method as query 6.
7     SELECT
8         t.client_id, t.instrument_id,
9         SUM(
10             (CASE WHEN t.side = 'BUY' THEN t.quantity ELSE -t.quantity END)
11             * COALESCE(ft.total_factor, 1.0) / COALESCE(
12                 (SELECT cf2.cumulative_factor FROM cumulative_factor cf2
13                     WHERE cf2.instrument_id = t.instrument_id AND cf2.date <= t."
14                     timestamp"
15                     ORDER BY cf2.date DESC LIMIT 1), 1.0)
16             ) AS calculated_position
17     FROM trades t
18     LEFT JOIN factor_total ft ON ft.instrument_id = t.instrument_id
19     GROUP BY t.client_id, t.instrument_id

```

```

18     HAVING ABS(calculated_position) > 0.0001
19 ),
20 last_reference_date AS (SELECT MAX(reference_date) AS reference_date FROM
    declared_positions),
21 discrepancies AS (
22     -- Pass 1: everything we calculated, matched (or not) to the declared side.
23     SELECT cp.client_id, cp.instrument_id, cp.calculated_position, dp.
        declared_quantity
24     FROM calculated_position cp
25     LEFT JOIN declared_positions dp
26         ON dp.client_id = cp.client_id AND dp.instrument_id = cp.instrument_id
27         AND dp.reference_date = (SELECT reference_date FROM last_reference_date)
28
29     UNION
30
31     -- Pass 2: everything declared, matched (or not) to what we calculated.
32     -- Combined with Pass 1 via UNION, this reproduces a FULL OUTER JOIN.
33     SELECT dp.client_id, dp.instrument_id, cp.calculated_position, dp.
        declared_quantity
34     FROM declared_positions dp
35     LEFT JOIN calculated_position cp ON cp.client_id = dp.client_id AND cp.
        instrument_id = dp.instrument_id
36     WHERE dp.reference_date = (SELECT reference_date FROM last_reference_date)
37 )
38 SELECT
39     d.client_id, c.name AS client_name, d.instrument_id, i.ticker,
40     ROUND(d.calculated_position, 4) AS calculated_position, d.declared_quantity,
41     ROUND(COALESCE(d.calculated_position, 0) - COALESCE(d.declared_quantity, 0),
        4) AS discrepancy,
42     CASE
43         WHEN d.calculated_position IS NULL THEN 'PHANTOM_AT_CUSTODIAN'
44         WHEN d.declared_quantity IS NULL THEN 'MISSING_AT_CUSTODIAN'
45         ELSE 'QUANTITY_MISMATCH'
46     END AS discrepancy_type
47 FROM discrepancies d
48 JOIN clients c     ON c.id = d.client_id
49 JOIN instruments i ON i.id = d.instrument_id
50 WHERE d.calculated_position IS NULL
51     OR d.declared_quantity IS NULL
52     OR ABS(d.calculated_position - d.declared_quantity) > 0.01
53 ORDER BY discrepancy_type, ABS(COALESCE(d.calculated_position, 0) - COALESCE(d.
    declared_quantity, 0)) DESC;

```

Listing 9: 07_reconciliation.sql — reconciliation of the calculated position against declared_positions.

Client	Ticker	Discrepancy	Type
Vantage Capital	ORBTE0	+11,954.0	MISSING_AT_CUSTODIAN
Harborview Holdings	ZENCA1	+5,130.5	MISSING_AT_CUSTODIAN
Vantage Capital	SILPL1	+3,160.0	MISSING_AT_CUSTODIAN
Harborview Holdings	AURIN2	+3,022.0	MISSING_AT_CUSTODIAN
Northbridge Fund	QUACO1	-117.1	PHANTOM_AT_CUSTODIAN
Thomas Garcia	QUACA3	-32.8	PHANTOM_AT_CUSTODIAN
Zoe Muller	BLUTE2	-11.5	PHANTOM_AT_CUSTODIAN
Northbridge Fund	REDHO9	-6.1	PHANTOM_AT_CUSTODIAN
Zoe Costa	SILPL1	+146.5	QUANTITY_MISMATCH
Northbridge Fund	ATLPA1	+118.2	QUANTITY_MISMATCH
Thomas Garcia	NORIN2	+116.5	QUANTITY_MISMATCH
Zoe Muller	SILPL1	-42.5	QUANTITY_MISMATCH
Alice Costa	FALTE3	+21.5	QUANTITY_MISMATCH
Alice Costa	HARIN3	-3.3	QUANTITY_MISMATCH

Table 3: All 14 breaks returned by Query 7 on the generated database: 4 MISSING_AT_CUSTODIAN, 4 PHANTOM_AT_CUSTODIAN, 6 QUANTITY_MISMATCH — exactly the 4 + 4 + 6 corruption recipe of Section 3.5, with no false positive and no miss.

4.7 Query 8 — Sector concentration (multi-level CTE aggregation)

Technique 7. A two-level CTE aggregation — per client-and-sector exposure, then per client total — computes each client’s sector-exposure percentages without ever materializing an intermediate portfolio total in Python: the denominator is itself just another grouped SUM() joined back in. Positions are valued at the latest close price using the corporate-action- adjusted quantity (same recursive-CTE logic as Query 6), since a stale, unadjusted quantity would misstate exposure for any client holding a stock that has since split. A client is flagged when a single sector exceeds 40% of their total portfolio value.

```

1 WITH RECURSIVE ca_ordered AS ( /* as in Query 5 */ ),
2 cumulative_factor AS ( /* as in Query 5 */ ),
3 factor_total AS ( /* as in Query 6 */ ),
4 calculated_position AS (
5     SELECT
6         t.client_id, t.instrument_id,
7         SUM(
8             (CASE WHEN t.side = 'BUY' THEN t.quantity ELSE -t.quantity END)
9             * COALESCE(ft.total_factor, 1.0) / COALESCE(
10                 (SELECT cf2.cumulative_factor FROM cumulative_factor cf2
11                     WHERE cf2.instrument_id = t.instrument_id AND cf2.date <= t."
12                         timestamp"
13                     ORDER BY cf2.date DESC LIMIT 1), 1.0)
14             ) AS calculated_position
15     FROM trades t
16     LEFT JOIN factor_total ft ON ft.instrument_id = t.instrument_id
17     GROUP BY t.client_id, t.instrument_id
18     HAVING calculated_position > 0.0001 -- only long, still-open positions
19     count towards exposure
20 ),
21 current_market_value AS (
22     SELECT instrument_id, close_price FROM (
23         SELECT instrument_id, close_price,
24             ROW_NUMBER() OVER (PARTITION BY instrument_id ORDER BY "timestamp"
25                 " DESC) AS rn
26         FROM market_data

```

```

24     ) WHERE rn = 1
25 ),
26 valued_position AS (
27     SELECT cp.client_id, i.sector, cp.calculated_position * cmv.close_price AS
           position_value
28     FROM calculated_position cp
29     JOIN instruments i          ON i.id = cp.instrument_id
30     JOIN current_market_value cmv ON cmv.instrument_id = cp.instrument_id
31 ),
32 sector_exposure AS (
33     SELECT client_id, sector, SUM(position_value) AS sector_value
34     FROM valued_position GROUP BY client_id, sector
35 ),
36 client_totals AS (
37     SELECT client_id, SUM(sector_value) AS total_portfolio_value FROM
           sector_exposure GROUP BY client_id
38 )
39 SELECT
40     se.client_id, c.name AS client_name, c.client_type, se.sector,
41     ROUND(se.sector_value, 2) AS sector_value,
42     ROUND(ct.total_portfolio_value, 2) AS total_portfolio_value,
43     ROUND(100.0 * se.sector_value / ct.total_portfolio_value, 2) AS
           pct_of_portfolio,
44     CASE WHEN se.sector_value / ct.total_portfolio_value > 0.40 THEN 1 ELSE 0
           END AS concentration_flag
45 FROM sector_exposure se
46 JOIN client_totals ct ON ct.client_id = se.client_id
47 JOIN clients c        ON c.id = se.client_id
48 ORDER BY se.client_id, pct_of_portfolio DESC;

```

Listing 10: 08_sector_concentration.sql — sector exposure and concentration flagging.

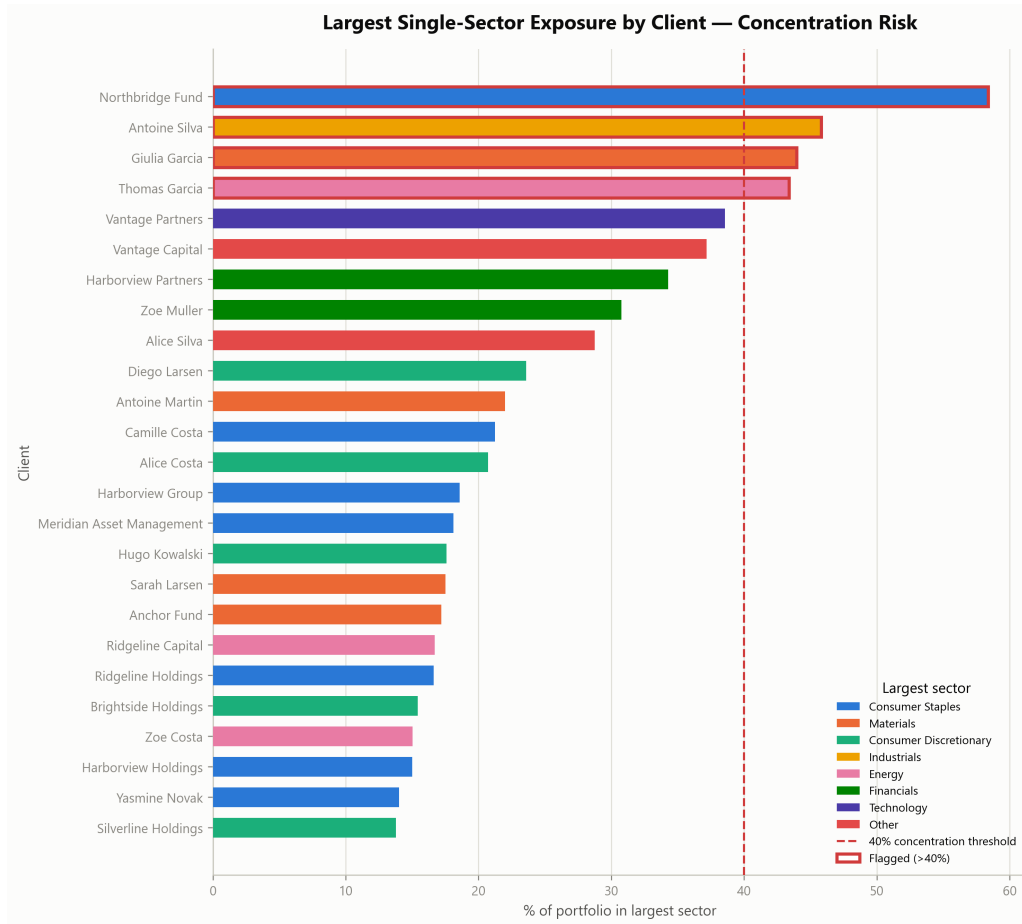


Figure 3: Each client’s largest single-sector exposure against the 40% threshold. Four clients are flagged: Northbridge Fund (58.4% Consumer Staples), Antoine Silva (45.8% Industrials), Giulia Garcia (44.0% Materials), and Thomas Garcia (43.4% Energy). Deliberately not a stacked full breakdown: stacking would put every segment but the first at an arbitrary offset, making a fixed threshold line meaningless for the rest.

4.8 Query 9 — Client performance ranking (RANK vs. DENSE_RANK)

Technique 8. Portfolio return is defined as total P&L (realized + unrealized, same moving-average-cost method as Query 2, aggregated per client rather than per client/instrument) divided by total capital deployed (the gross sum of BUY trade amounts). `RANK()` and `DENSE_RANK()` are computed side by side over the same ordering to make their tie-breaking difference explicit: `RANK()` leaves gaps after ties (appropriate for “your rank among N clients”), `DENSE_RANK()` does not (appropriate for “how many distinct performance tiers are above you”).

```

1 WITH trade_costing AS ( /* as in Query 2, unaggregated by instrument */ ),
2 average_cost AS ( /* as in Query 2 */ ),
3 realized_pnl_by_client AS (
4     SELECT client_id,
5         SUM(CASE WHEN side = 'SELL' THEN (price - avg_acquisition_cost) *
6             quantity ELSE 0 END) AS realized_pnl
7     FROM average_cost GROUP BY client_id
8 ),
9 last_position AS ( /* as in Query 2 */ ),
10 current_market_value AS ( /* as in Query 2 */ ),
11 unrealized_pnl_by_client AS (
12     SELECT lp.client_id, SUM((cmv.close_price - lp.avg_acquisition_cost) * lp.
13         open_position) AS unrealized_pnl

```

```

12     FROM last_position lp
13     JOIN current_market_value cmv ON cmv.instrument_id = lp.instrument_id
14     WHERE lp.open_position > 0.0001
15     GROUP BY lp.client_id
16 ),
17 capital_deployed AS (
18     SELECT client_id, SUM(quantity * price) AS total_invested
19     FROM trades WHERE side = 'BUY' GROUP BY client_id
20 ),
21 client_performance AS (
22     SELECT
23         cd.client_id, cd.total_invested,
24         COALESCE(rp.realized_pnl, 0) + COALESCE(up.unrealized_pnl, 0) AS
            total_pnl,
25         (COALESCE(rp.realized_pnl, 0) + COALESCE(up.unrealized_pnl, 0)) / cd.
            total_invested AS portfolio_return
26     FROM capital_deployed cd
27     LEFT JOIN realized_pnl_by_client rp ON rp.client_id = cd.client_id
28     LEFT JOIN unrealized_pnl_by_client up ON up.client_id = cd.client_id
29 )
30 SELECT
31     p.client_id, c.name AS client_name, c.client_type,
32     ROUND(p.total_invested, 2) AS total_invested, ROUND(p.total_pnl, 2) AS
        total_pnl,
33     ROUND(p.portfolio_return * 100, 3) AS return_pct,
34     RANK() OVER (ORDER BY p.portfolio_return DESC) AS rank,
35     DENSE_RANK() OVER (ORDER BY p.portfolio_return DESC) AS dense_rank
36 FROM client_performance p
37 JOIN clients c ON c.id = p.client_id
38 ORDER BY rank;

```

Listing 11: 09_performance_ranking.sql — client performance ranking by portfolio return.

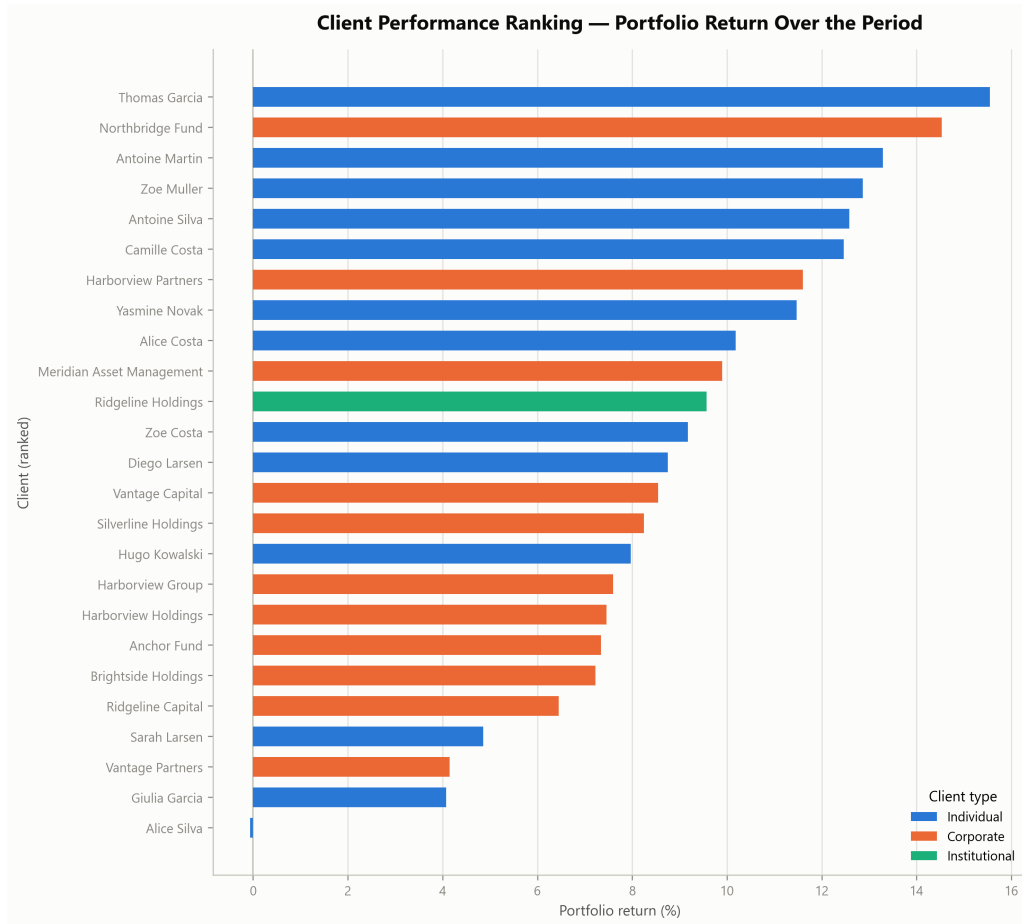


Figure 4: Client performance ranking, 25 clients. Thomas Garcia leads at +15.55% return; Alice Silva is the only client with a negative return (−0.065%) over the period.

5 SQLite Gaps Closed in Pure SQL

SQLite deliberately ships a minimal SQL surface. Rather than dropping to `pandas` whenever a built-in was missing, every gap below is closed with a documented, standard SQL pattern that stays inside the database engine:

- **No PERCENTILE_CONT/PERCENTILE_DISC** (no ordered-set aggregate `WITHIN GROUP` syntax at all) → Query 3 sorts returns with `ROW_NUMBER()` and selects the rank matching the target percentile (Section 4.3).
- **No STDEV/STDEV_POP window function** → Query 4 derives it manually from $\text{AVG}(x^2) - \text{AVG}(x)^2$, guarded with `MAX(variance, 0)` against floating-point rounding (Section 4.4).
- **No native forward-fill** for “position as of any calendar date” → Query 3 unions trade events with zero-delta calendar markers, using an `is_marker` discriminator in the `ORDER BY` to guarantee same-day trades settle first, and reads a running `SUM()` back out instead of one correlated subquery per calendar day.
- **No FULL OUTER JOIN** → Query 7 uses the classic two-`LEFT JOIN`-plus-`UNION` emulation (Section 4.6).

6 Query Optimization: EXPLAIN QUERY PLAN and Measured Timing

The three heaviest queries in this project — Query 1, Query 4, and Query 3 — each run a window function over the full `trades` table, requiring rows already sorted by (`partition columns...`, `timestamp`). As stated in Section 2, `generate_data.py` intentionally builds the database with *no* secondary indexes, so that `analysis.py` can measure the real before/after effect of adding them.

Technique 9. Without a matching index, SQLite must materialize the partition/order key into a temporary B-tree to sort it before the window function can stream through it — visible in `EXPLAIN QUERY PLAN` as `USE TEMP B-TREE FOR ORDER BY`. A composite index matching (`PARTITION BY...`, `ORDER BY...`) lets SQLite scan the table already in the required order, and that plan line disappears — e.g. Query 1’s plan changes from `SCAN t` to `SCAN t USING INDEX idx_trades_client_instrument_ts`.

Query	Median before	Median after	Gain
Query 1 — running-total position	0.1396 s	0.0940 s	32.7%
Query 4 — z-score anomaly detection	0.0780 s	0.0460 s	41.1%
Query 3 — historical VaR	1.3631 s	1.3228 s	3.0%

Table 4: Measured result on `data/risk.engine.db`, 7 runs each, median wall-clock via `time.perf_counter()` (`results/optimization_comparison.csv`). Query 3 benefits both from the `trades` index (removing the sort feeding its `UNION ALL` branch) and from `idx_market_data_instrument_ts` (the `calendar/price` join becomes a covering-index scan instead of hitting the implicit rowid-based primary-key index).

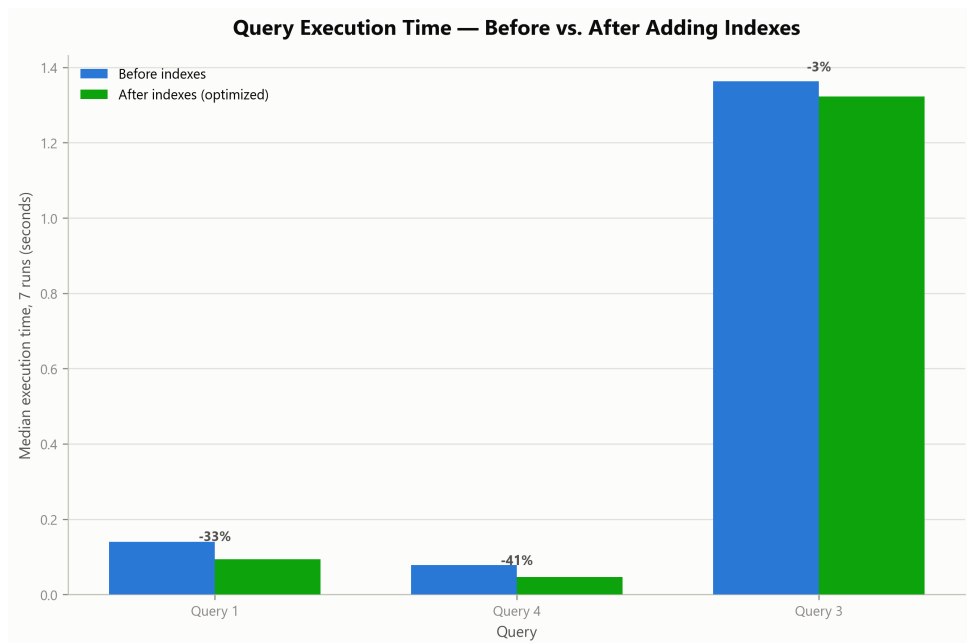


Figure 5: Median query execution time before vs. after adding the four composite indexes of Section 2.

Honest limitation 1. Run-to-run variance on a single machine is normal; repeated runs of these three queries can land anywhere in a wide range depending on system load. The deterministic

part of the story is the *query-plan* change (the temporary-sort step disappearing), not the specific percentage measured on any one run.

Honest limitation 2. At $\approx 42,000$ trade rows and $\approx 21,000$ `market.data` rows, SQLite’s whole working set sits comfortably inside the OS page cache, so the *absolute* wall-clock gain measured here is modest even though the query plan is structurally cheaper (an $O(n \log n)$ sort removed from the plan). Query 3’s gain is markedly smaller (3.0%) than Query 1’s or Query 4’s (32.7%, 41.1%) precisely because most of its cost is the $O(\text{clients} \times \text{instruments} \times 250)$ calendar cross-join and portfolio aggregation, not the part the new indexes touch. At realistic production row counts the eliminated sort would dominate far more and the relative gain would widen considerably for Queries 1 and 4; the query-plan change is the reliable signal, the millisecond figure in Table 4 is not.

7 Report Exports: Figures and Result Tables

`analysis.py` writes two sets of report-ready artifacts, both derived exclusively from SQL result sets — `matplotlib` and `pandas` format and plot the numbers; they never recompute them. `figures/*.png` (300 dpi, tight bounding box) contains the five figures referenced throughout Section 4 and Section 6. `results/*.csv` contains one file per query result set, listed in Table 5.

File	Source query	Rows
<code>positions_running_total.csv</code>	Query 1	42,000
<code>pnl_realized_unrealized.csv</code>	Query 2	1,000
<code>var_by_client.csv</code>	Query 3	25
<code>detected_anomalies.csv</code>	Query 4	546
<code>corporate_action_factors.csv</code>	Query 5	9
<code>positions_adjusted.csv</code>	Query 6	42,000
<code>position_reconciliation.csv</code>	Query 7	14
<code>sector_concentration.csv</code>	Query 8	250
<code>performance_ranking.csv</code>	Query 9	25
<code>optimization_comparison.csv</code>	Section 6	3

Table 5: CSV exports, one per query result set plus the optimization summary, with actual row counts from the generated database.

8 Honest Limitations and Scope

In the same spirit as stating finite-size effects alongside an asymptotic theoretical result, this section lists what this project does *not* claim.

Honest limitation 3. Query 4’s anomaly detector scores *raw* trade prices. As shown in Section 4.4, the single most extreme z-score in the dataset is not a genuine price dislocation but the mechanical price jump on MERIN1’s reverse-split date. A production version would need to adjust prices by the same recursive corporate-action factor used in Queries 5–6 before scoring them, or exclude known corporate-action dates from the rolling window entirely.

Honest limitation 4. Query 9’s “portfolio return” is total P&L divided by gross capital deployed (the sum of BUY amounts, never netted for sells and not time-weighted). It is a simple, transparent metric, but it is not a money-weighted (IRR-style) or time-weighted return, and it will treat a client who bought once and held very differently from one who bought and sold the same notional repeatedly, even at an identical realized/unrealized outcome.

Honest limitation 5. The reconciliation tolerance in Query 7 ($\text{ABS}(\text{difference}) > 0.01$) is a fixed absolute threshold chosen to match the scale of the synthetic data; a production reconciliation engine would typically use a relative tolerance, or an instrument-specific one tied to lot size.

Honest limitation 6. This is a single-node SQLite project by design — it demonstrates SQL technique, not distributed query engines, and the optimization results of Section 6 should be read as a query-plan case study, not as a general benchmark of index performance at scale.

Honest limitation 7. All row counts and reconciliation-break counts quoted in this document are reproducible exactly given `RANDOM_SEED = 42`, but the specific wall-clock timings in Table 4 are machine-dependent and are reported as measured on one run rather than a guaranteed constant.

9 Conclusion

Across nine queries against a single SQLite database, this project demonstrates: running-total and forward-fill positions via window functions; multi-stage moving-average-cost P&L via eight chained CTEs; historical VaR via event-time forward-fill and a manually derived percentile; anomaly detection via a manually derived rolling standard deviation — along with an honest account of that detector’s blind spot around corporate actions; a recursive CTE composing an ordered sequence of corporate actions into a cumulative adjustment factor, including a reverse split and a genuine two-factor compounding case; a portable `FULL OUTER JOIN` emulation that recovers all 14 deliberately injected reconciliation breaks with no false positive; multi-level CTE aggregation for concentration risk; and `RANK()` versus `DENSE_RANK()` tie-breaking semantics — together with a query-plan-driven optimization exercise that measures, rather than assumes, the effect of matching an index to a window function’s partition and order columns (32.7% and 41.1% median gains on Queries 1 and 4, a modest 3.0% on Query 3 for reasons explained in Section 6). The skill set exercised here — reading a query plan, closing a missing statistical primitive inside the database rather than outside it, and reasoning honestly about what a measured result does and does not generalize to — maps directly onto the kind of SQL fluency expected in quantitative-risk and data-engineering roles.

A Full Database Schema

```

1 CREATE TABLE IF NOT EXISTS instruments (
2     id            INTEGER PRIMARY KEY,
3     ticker        TEXT      NOT NULL UNIQUE,
4     name          TEXT      NOT NULL,
5     sector        TEXT      NOT NULL,
6     asset_class   TEXT      NOT NULL CHECK (asset_class IN ('Equity', 'Bond', 'ETF
7     currency      TEXT      NOT NULL CHECK (currency IN ('EUR', 'USD', 'GBP', 'CHF
8 );
9
10 CREATE TABLE IF NOT EXISTS clients (
11     id            INTEGER PRIMARY KEY,
12     name          TEXT      NOT NULL,
13     client_type   TEXT      NOT NULL CHECK (client_type IN ('Individual', '
14     opening_date  TEXT      NOT NULL
15 );
16
17 CREATE TABLE IF NOT EXISTS trades (
18     id            INTEGER PRIMARY KEY,
19     client_id     INTEGER NOT NULL REFERENCES clients(id),

```

```

20     instrument_id    INTEGER NOT NULL REFERENCES instruments(id),
21     "timestamp"      TEXT      NOT NULL,
22     quantity         REAL      NOT NULL CHECK (quantity > 0),
23     price            REAL      NOT NULL CHECK (price > 0),
24     side             TEXT      NOT NULL CHECK (side IN ('BUY', 'SELL'))
25 );
26
27 CREATE TABLE IF NOT EXISTS market_data (
28     instrument_id    INTEGER NOT NULL REFERENCES instruments(id),
29     "timestamp"      TEXT      NOT NULL,
30     close_price       REAL      NOT NULL CHECK (close_price > 0),
31     volume           INTEGER NOT NULL CHECK (volume >= 0),
32     PRIMARY KEY (instrument_id, "timestamp")
33 );
34
35 CREATE TABLE IF NOT EXISTS corporate_actions (
36     id               INTEGER PRIMARY KEY,
37     instrument_id    INTEGER NOT NULL REFERENCES instruments(id),
38     date             TEXT      NOT NULL,
39     action_type      TEXT      NOT NULL CHECK (action_type IN ('SPLIT', 'DIVIDEND'))
40     ,
41     ratio            REAL      NOT NULL
42 );
43
44 CREATE TABLE IF NOT EXISTS declared_positions (
45     client_id        INTEGER NOT NULL REFERENCES clients(id),
46     instrument_id     INTEGER NOT NULL REFERENCES instruments(id),
47     reference_date    TEXT      NOT NULL,
48     declared_quantity REAL      NOT NULL,
49     PRIMARY KEY (client_id, instrument_id, reference_date)
50 );
51
52 -- Optimized indexes (applied live by analysis.py; see Section 6)
53 CREATE INDEX IF NOT EXISTS idx_trades_client_instrument_ts
54     ON trades (client_id, instrument_id, "timestamp");
55 CREATE INDEX IF NOT EXISTS idx_trades_instrument_ts
56     ON trades (instrument_id, "timestamp");
57 CREATE INDEX IF NOT EXISTS idx_corporate_actions_instrument_date
58     ON corporate_actions (instrument_id, date);
59 CREATE INDEX IF NOT EXISTS idx_market_data_instrument_ts
60     ON market_data (instrument_id, "timestamp");

```

Listing 12: schema.sql — baseline tables and optimized indexes.

B Reproducing the Results

```

1  pip install numpy pandas matplotlib
2  python generate_data.py    # builds data/risk_engine.db (baseline schema, no
                             # indexes)
3  python analysis.py        # runs all 9 queries, the optimization section,
                             # then exports figures/*.png and results/*.csv
4

```

Listing 13: Reproducing every number and figure in this document.